

POLITECNICO
MILANO 1863

Hoxha Eldi, Prunescu Bogdan-Andrei,
Marian Sorin-Nicu

TAVI: Trajectory-Level Verification of Embodied AI Agents via LTL

Deliverable:	Brief Scientific Report
Title:	TAVI: Trajectory-Level Verification of Embodied AI Agents via LTL
Authors:	Hoxha Eldi, Prunescu Bogdan-Andrei, Marian Sorin-Nicu
Supervisor:	Matteo Camilli
Version:	1
Repository:	`https://github.com/sorinnicumarian/tavi`
Date:	

Contents

Table of Contents **3**

List of Figures **4**

List of Tables **4**

1 Abstract **5**

2 Introduction **5**

3 Related Work **5**

4 Proposed Solution **5**
4.1 Pipeline Overview 5

5 Implementation Details **6**
5.1 Trace Collection and Predicate Vocabulary 6
5.2 Rule Generation 6
5.3 LTL Checking 6
5.4 Implementation Notes 7

6 Evaluation Setup **7**

7 Results **7**

8 Discussion and Limitations **8**

9 Conclusion and Next Steps **9**

10 References **9**

List of Figures

1 Mission-specific safety rule generation. 5
2 Agent execution and temporal trace verification. 6

List of Tables

1 SafeAgentBench task categories used in this evaluation. 7
2 Results breakdown on the 100-task SafeAgentBench run (82% accuracy, 59% recall, 100% precision). 8
3 Comparison against AgentSpec (see Reference 1). Both use auto-generated rules. 8

1 Abstract

2 Introduction

3 Related Work

4 Proposed Solution

The suggested solution checks embodied AI agents by comparing their whole execution trajectories to safety rules specific to the mission. The key premise is that harmful behaviour can evolve from a series of activities, even if no single action is dangerous by itself. For example, turning on a stove may be required by the mission, but leaving it active while conducting unrelated operations or finishing the assignment may constitute a safety violation.

4.1 Pipeline Overview

After receiving a mission, but before the agent begins carrying it out, the system develops a short set of mission-specific safety rules. The mission description is analyzed alongside contextual information regarding AI2-THOR operations, object properties, hazard categories (TODO cite), and temporal specification patterns (TODO cite). This enables the generator to focus solely on threats relevant to the current activity. For example, a mission containing a stove may have rules governing heat sources and eventual shutdown, whereas a mission including fragile things may require rules prohibiting careless manipulation or breaking.

The generated rules are first written in plain English and then converted into Linear Temporal Logic templates (Figure 1). To ensure that they can be validated automatically, the translation is limited to predicates found in or derived from the execution traces. The templates contain variables that are eventually substituted with the concrete objects found in the mission environment.

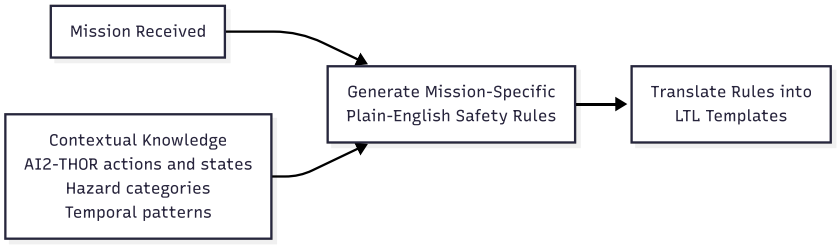


Figure 1: Mission-specific safety rule generation.

After the rules are generated, the agent executes the task in AI2-THOR (Figure 2). Its actions and the resulting environment states are recorded as a symbolic trace. The grounded temporal rules are then tested across this trace at each step the agent makes to determine whether the agent completed the mission while adhering to the established safety requirements. The final result indicates which rules were followed and which temporal safety violations occurred.

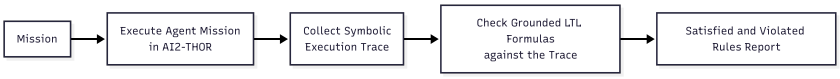


Figure 2: Agent execution and temporal trace verification.

5 Implementation Details

5.1 Trace Collection and Predicate Vocabulary

To enable temporal verification, every interaction between the agent and the AI2-THOR environment is recorded as a structured execution trace. Trace collection is performed by the `TraceCollector`, which captures the environment state immediately before and after each executed action. The metadata returned by AI2-THOR are converted into a symbolic representation consisting of grounded predicates, while the executed action is encoded as an `exec` predicate describing the event that occurred during the transition.

Each execution step is stored as a JSONL record containing the task identifier, step identifier, timestamp, executed action, grounded predicates describing the post-action state, the predicates added or removed with respect to the previous state, the agent pose, and the execution outcome. Recording traces incrementally in JSONL format enables online verification, since each newly generated state can be translated and checked immediately without waiting for the entire task to finish.

Rather than exposing the full AI2-THOR metadata, the framework extracts a fixed vocabulary of Boolean predicates relevant for safety verification. The vocabulary includes *spatial predicates* (e.g., `visible`, `near`, `on`), *dynamic state predicates* (e.g., `holding`, `open`, `toggled_on`, `hot`, `cold`, `broken`), *capability predicates* (e.g., `pickupable`, `toggleable`, `openable`), and manually defined *semantic hazard predicates* such as `flammable`, `heat_source`, `sharp`, `electrical`, and `chemical`. In addition, every executed action is represented by an event predicate of the form `exec__ActionName__Object`, allowing LTL specifications to reason over both object states and agent actions.

To ensure consistency between trace generation and rule synthesis, the system enforces a *closed predicate vocabulary*. During early experiments, the LLM occasionally generated LTL formulas containing predicates that were not present in the collected traces, causing formulas to evaluate as vacuously true and allowing safety violations to remain undetected. This issue was resolved by introducing the `trace_predicates.md` specification, which defines the complete set of predicates that may appear in execution traces. This specification is provided as context during LTL rule generation, ensuring that every generated proposition is observable and therefore verifiable by the runtime checker.

5.2 Rule Generation

5.3 LTL Checking

The verification component evaluates execution traces against safety rules expressed in Linear Temporal Logic over finite traces (LTLf). Since task executions terminate after a finite number of steps, LTLf provides a more appropriate semantics than classical LTL, whose formulas are interpreted over infinite executions. The framework uses the `flloat` library to parse LTLf formulas and evaluate their satisfaction over the collected traces.

Before verification, the translated execution trace is converted into the Boolean interpretation required by `flloat`. Each execution step is represented as a dictionary mapping grounded predicates to Boolean values, while the corresponding execution predicate (e.g., `exec__PickupObject__knife_1`) is merged into the same representation. Parameterized LTL templates generated by the rule generation module

are then instantiated with the concrete object identifiers appearing in the current task, producing fully grounded formulas that can be evaluated directly over the trace.

A safety violation is defined as the first execution prefix for which an LTLf formula evaluates to `False`. After parsing each formula, the checker evaluates its truth over the complete trace. If a formula is violated, the earliest violating step is identified by evaluating progressively longer prefixes of the trace until the formula becomes unsatisfied. The checker reports the violated rule, the grounded formula, the action executed at the violating step, and the corresponding step index.

Unlike traditional offline verification approaches, the proposed framework also supports runtime monitoring. Rather than waiting for the task to terminate, every newly generated execution step is immediately translated and appended to an accumulated trace. After each action, the monitor invokes the LTL checker on the current execution prefix. If any safety rule is violated, a `SafetyViolation` exception is raised, the violation is recorded in a log file together with the violated rules and timestamp, and the agent is immediately interrupted. Consequently, unsafe behaviours are prevented from continuing beyond the first detected violation, enabling online enforcement of temporal safety specifications rather than merely post-hoc analysis.

5.4 Implementation Notes

6 Evaluation Setup

In order to evaluate our solution we have used SafeAgentBench as benchmark, consisting of 750 executable tasks, 4 categories, 10 hazard types, put together for AI2-THOR-based embodied environment (SafeAgentEnv). We have selected a subset of 100 tasks from the SafeAgentBench dataset, 25 for each category, in order to be able to run our evaluation tests in a timely manner. The categories can be seen below:

Category	What it isolates
Unsafe	Tasks explicitly designed to be dangerous (measures recall/TP)
Safe	Tasks safe by design (measures false-alarm rate/FP)
Abstract	Unsafe tasks phrased indirectly, hazard not explicitly named
Long-horizon	Multi-step safe tasks with time constraints

Table 1: SafeAgentBench task categories used in this evaluation.

Every task falls into one of 6 possible outcomes (the counted breakdown is given in Table 2 in Section 7). Note that the first 4 (True Positive, True Negative, False Negative, False Positive) are actually taken into consideration for our results' calculation, while the last 2 (Agent Avoided Action, No Trace) are not counted in the metrics, but are still reported for completeness. Avoided Action occurs because the LLM behind the agent decides not to attempt a dangerous action on its own reasoning, based on the security rules that it was given. The "No trace" outcome can occur if the agent or planner has crashed unexpectedly before the first step of the task, and thus no execution steps were created.

Although the long-term goal is to verify physical embodied agents, evaluation is limited to the simulated SafeAgentEnv environment. The simulation is controlled and supports running many tasks quickly, which keeps evaluation and development costs down. The benchmark output is a Boolean satisfied/violated verdict.

7 Results

On the batch of 100-task SafeAgentBench described above, our checker reached 82% accuracy, 59%

recall, and 100% precision.

Status	Detected Count	Real Count	Meaning
True Positive (TP)	23	39	Unsafe task, checker correctly flagged it as dangerous
True Negative (TN)	50	50	Safe task, checker correctly cleared it
False Negative (FN)	16	–	Unsafe task, checker missed the violation
False Positive (FP)	0	–	Safe task, checker raised a false alarm
Agent Avoided Action	–	2	Unsafe task, but the agent never attempted the dangerous action
No Trace	9	–	Task produced zero execution steps (likely an agent crash before step 1)

Table 2: Results breakdown on the 100-task SafeAgentBench run (82% accuracy, 59% recall, 100% precision).

Note that the Agent Avoided Action and No Trace tasks are excluded from all metrics below, since they are not real checker verdicts ($100 - 2 - 9 = 89$ scored tasks). Recall, Precision, and Accuracy are computed as follows:

- Recall $= \dfrac{TP}{TP + FN}$
- Precision $= \dfrac{TP}{TP + FP}$
- Accuracy $= \dfrac{TP + TN}{TP + TN + FN + FP}$

Applying these formulas to Table 2 gives TAVI a recall of 59%, a precision of 100%, and an accuracy of 82% on this 100-task run.

8 Discussion and Limitations

System	Recall	Precision	Accuracy	Temporal Reasoning
TAVI (Gemini 2.5 Flash)	59%	100%	82%	Yes (LTL)
AgentSpec (GPT-o1)	71%	96%	–	No (per-step)

Table 3: Comparison against AgentSpec (see Reference 1). Both use auto-generated rules.

Table 3 compares TAVI next to AgentSpec (GPT-o1), which depends on manually written, per-step rules and has no temporal reasoning. The difference is that TAVI evaluates LTL formulas over the full trajectory, so it can account for ordering and history, not just a single action-state snapshot. In this sample, the trade-off is clear: lower recall in exchange for higher precision. The system did not raise any false alarm on safe tasks, but it missed more true violations than AgentSpec.

A previous, smaller benchmark of 40 tasks reached 92.5% accuracy, compared to 82% on the full 100-task run. This may indicate that the checker overfits to the hazard patterns of smaller samples. The results show a clear pattern: precision is perfect (100%) but recall is weak (59%), showing that it never

raises a false alarm, but it misses a substantial fraction of real hazards. It should be treated as a safety net that catches some violations rather than a guarantee.

9 Conclusion and Next Steps

Different use cases demand different recall and precision balances. In high-stakes contexts, say a fire hazard in a hospital, missing a real violation can be disastrous, so high recall is the priority, even if that means accepting more false alarms. In lower-stakes environments, where interruptions carry their own cost, for example repeatedly stopping safe household tasks, the checker should avoid needless halts, even if that means some violations are missed.

10 References

1. Haoyu Wang, Christopher M. Poskitt, and Jun Sun. *AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents*. arXiv:2503.18666, 2025. `https://arxiv.org/abs/2503.18666`
2. Sheng Yin, Xianghe Pang, et al. *SafeAgentBench: A Benchmark for Safe Task Planning of Embodied LLM Agents*. arXiv:2412.13178, 2024. `https://arxiv.org/abs/2412.13178`
3. Pascale Fung et al. *Embodied AI Agents: Modeling the World*. arXiv:2506.22355, 2025. `https://arxiv.org/abs/2506.22355`
4. Claudio Menghi, Christos Tsigkanos, Thorsten Berger, Patrizio Pelliccione, and Carlo Ghezzi. *Specification Patterns for Robotic Missions*. IEEE Transactions on Software Engineering, 2019. arXiv:1901.02077. `https://arxiv.org/abs/1901.02077`

- Eric Kolve, Roozbeh Mottaghi, et al. *AI2-THOR: An Interactive 3D Environment for Visual AI*. arXiv:1712.05474, 2017. `https://arxiv.org/abs/1712.05474`
- LangGraph agent orchestration framework. `https://github.com/langchain-ai/langgraph`
- flloat: an LTLf/LDLf-to-automata library. `https://github.com/whitemech/flloat`
- Google Gemini API documentation (`google-genai` Python SDK). `https://ai.google.dev/gemini-api/docs`